1) Quantization

The quantization is the processo of mapping a function Q from $\mathbb{R}$ to a discrete set called *Dictionary*.

$$Q : x \in \mathbb{R} \to y \in C = \{\hat{x}_1, \hat{x}_2, \dots\} \subset \mathbb{R}$$

where

- C is the Dictionary a subset of $\mathbb{R}$
- $\hat{x}_i$ quantization level (code-word)
- $e = x - Q(x)$: The quantization noise (error)
- $\Theta_i = \{x : Q(x) = \hat{x}_i\}$ the decision region. Defined as non intersecting intervals

Quantization can be seen as an encoding/decoding process:
take signal $x(n)$: the encoding step assigns each sample of $x(n)$ to a quantization level $i(n)$. The decoder associates to each quantization level $i(n)$ the corresponding code word (value).

Performance criteria are composed of **Rate** R and **distortion** D.
By assumption $\boxed{R = \log_2 L}$, which is the

- The rate R is the avg number of bits needed to represent a sample. We define

$$\boxed{R = \log_2 L}$$

- The distortion D is the MSE which in case of a random signal equals to the variance:

$$D = \mathbb{E}[|x(n) - Q(x(n))|^2] = \sigma_Q^2$$

1.1) Uniforma Quantization (UQ)

In uniforma quantizers the input range is divided into $L = 2^b$ equal-sized cells with size $\Delta = A/L$. Each cell is represented by its mid-point i.e. the levels are the centers of the cells.

$$\begin{gathered}
\forall i,\ \Delta^i = \Delta = A/L \\
t^i = t^{i-1} + \Delta \\
\hat{x}^i = \frac{t^i + t^{i-1}}{2} \\
\Theta^i = \left(\hat{x}^i - \frac{\Delta}{2}, \hat{x}^i + \frac{\Delta}{2}\right)
\end{gathered}$$

Unsigned Data

In this case the data is in (a, A) and the ceil function suffices. The thresholds are $(0, \Delta, 2\Delta, \dots, L\Delta)$ and thus $t^i = (i-1)\Delta$.

$$\begin{aligned}
&\text{Encoder: } i = \left\lceil \frac{x}{2} \right\rceil \\
&\text{Decoder: } \hat{x}^i = i\Delta - \frac{\Delta}{2} \\
&\implies Q(x) = \Delta \left\lceil \frac{x}{2} \right\rceil - \frac{\Delta}{2}
\end{aligned}$$

Unsigned Data

In this case the data is in $(-A/2, A/2)$ and the round function is used. Moreover since 0 must be a quantizer value not a threshold a **odd number of levels is required**

$$\text{Encoder: } i = \text{round}\left(\frac{x}{2}\right)$$
$$\text{Decoder: } \hat{x}^i = i\Delta$$
$$\implies Q(x) = \Delta\text{round}\left(\frac{x}{2}\right)$$

This is called a **mid-tread** quantizer which keeps near 0 values (noise) to 0.

1.2) Rate Distortion (RD) Curve

First recall the uniform distribution:

$$\mathcal{X} \sim u(a,b), \quad f_X(x) = \begin{cases} \frac{1}{b-a} & x \in [a,b] \\ 0 & \text{elsewhere} \end{cases}, \quad \mathbb{E}[\mathcal{X}] = \frac{b-a}{2}, \quad \text{var}(\mathcal{X}) = \frac{(b-a)^2}{12}$$

And the Law of the Unconscious Statistician (LOTUS):

$$E[g(x)] = \int g(x) f_X(x) dx$$

These allows to calculate the distortion of a uniform RV quantized with UQ:

Let $X \sim U\left(-\frac{A}{2}, \frac{A}{2}\right)$ and $Q(x)$ a UQ with L levels. Then:

$$D = \sigma_Q^2 = \mathbb{E}[(X - \hat{X})^2] = \frac{\Delta^2}{12} = \frac{1}{12}\frac{A^2}{L^2} = \sigma_x^2 2^{-2R}$$

$$SNR = 10\log_{10}\frac{\mathbb{E}[X^2]}{D} = 10\log_{10}2^{2R} \approx 6R$$

Proof:
todo calculation of interval
From here recall that $R = \log_2 L \to L = 2^R$ and that since X is uniform it has variance $A^2/12$ and thus $\frac{A^2}{12L^2} = \sigma_x^2/2^{2R}$.
The SNR is very straight forward: Notice $\mathbb{E}[X^2] = \sigma_x^2$

High Resolution (HR) Quantization

For high resolution we mean $L \to \infty$
For a region Θ^i the value p_x is constant.
The quantization noise in Θ^i is $\sim u\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right)$ and with the total probability law we have

$$E \sim u\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right) \to D = \frac{\Delta^2}{12} = \frac{A^2}{12}2^{-2R}$$

and the SNR becomes

$$SNR = 10\log_{10}\frac{\mathbb{E}[X^2]}{D} = 10\log_{10}\frac{\sigma_X^2}{A^2/12}2^{2R} \approx 6R - 10\log_{10}\frac{A^2/4}{3\sigma_X^2} = 6R - 10\log_{10}\frac{\gamma^2}{3}$$

where γ^2 is the load factor, that is, the ratio between peak power and avg power. With this in mind:

$$D = \frac{\gamma^2}{3}\sigma_x^2 2^{-2R} = K_X \sigma_X^2 2^{-2R}$$

1.3) Optimal Scalar Quantization (SQ)

For a given probability density function $p_X(x)$ we want to find the quantizer (thresholds t^i and levels $\hat{x}^i$) that minimizes the distortion D for a given rate R

Optimal HR Quantizer

Recall the HR hypothesis:

$$L \to \infty \quad \max_i \Delta_i \to 0 \quad \forall i, u \in \Theta^i,\ p_X(x) \approx P_i$$

The RD curve becomes:

$$\sigma_Q^2 = c_X \sigma_X^2 2^{-2R} \quad \text{with } c_X = \frac{1}{12}\left[\int p_U^{1/3}(t)dt\right] \text{ and } U = \frac{X}{\sigma_X}$$

c_X is the shape factor since it only depends on the pdf.
some common shape factors are:

- Uniform: $c_X = 1$
- Gaussian $c_X = \frac{\sqrt{3}}{2}\pi \approx 2.72$

Non HR Quantizer

The solution doesn't have closed analytical formula, but

- There exists necessary conditions
- Lloyd-Max algorithm for local minima

How does the algorithm work?

1. Choose initial dictionary $C^{(0)} = \{\hat{x}_0^i\}_{i=1,\dots,L}$
2. Find best thresholds for given dictionary $\left\{t^i = \frac{\hat{x}^i + \hat{x}^{i+1}}{2}\right\}$
3. Find best dictionary for given thresholds $C^{(1)} = \{\hat{x}_{(1)}^i\}$
4. iterate 2. 3. until convergence (stop condition)

The initial dictionary can be chosen:

- Uniform Initialization: $\hat{x}^n = -A_0 + n\Delta\ \forall n = 1, \dots, L$
- Random initialization: code-words are initialized with the same distribution as data

The (quantizer) decision rule is the nearest neighbor:

$$k = \arg\min_n |x - \hat{x}^n| \to Q(x) = \hat{x}^k$$

The threshold decision rule is:

$$t^i = \frac{\hat{x}^i + \hat{x}^{i+1}}{2}$$

1.4) Predictive SQ

Quantization alone is not effective for compression of non-sparse data. Often samples have dependencies between each other (image has many adjecent pixels of same color).

This algorithm exploits the sample correlationand reduces the variance by increasing sparsity
But what is a sparse signal?

Definition 1 (Sparse Signal)**.**

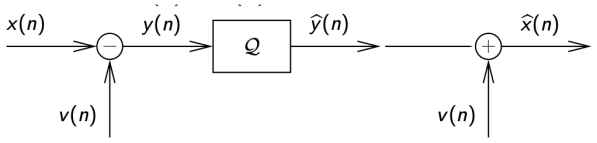
A signal is sparse if most of its components are zero or close to zero

- The variance of a sparse signal is low

- Zero (or close to zero) samples can be neglected (quantized to 0) without introducing distortion
- Natural signal are not sparse but can easily be transformed into one

The basic idea is to define the prediction error as the signal error

$$q(n) = y(n) - \hat{y}(n) = x(n) - v(n) - (\hat{x}(n) - v(n)) = \overline{q}(n)$$



Quantizer process

So the SNR can be written as a sum of two terms:

$$SNR = 10\log_{10}\frac{\sigma_X^2}{D} = 10\log_{10}\frac{\sigma_X^2}{\sigma_Y^2} + 10\log_{10}\frac{\sigma_Y^2}{D} = G_P + G_Q$$

Where G_Q is the standard quantizer SNR on the signal y while G_P is the term added which is positive only if the variance of the sparse signal y is lower than the og signal x

Example 2.

Consider a gaussian signal $X(n) \sim \mathcal{N}(0, \sigma^2)$ and $V(n) = X(n-1)$. Moreover suppose that $\mathbb{E}[X(n)X(m)] = \sigma^2\rho^{|n-m|}$. The sparse signal becomes

$$Y(n) = X(n) - X(n-1)$$

The variance moreover becomes:

$$\begin{aligned}\sigma_Y^2 &= \mathbb{E}[(X(n) - X(n-1))^2] \\ &= \mathbb{E}[X(n)^2 + X(n-1)^2 - 2X(n)X(n-1)] \\ &= 2\sigma^2 - 2\sigma^2\rho\end{aligned}$$

and the prediction gain becomes

$$G_p = 10\log_{10}\frac{\sigma^2}{2(1-\rho)\sigma^2}$$

which is good only if $G_P > 0 \to \rho > \frac{1}{2}$

Linear Predictor

A linear predictor is a linear combination of the P previous values

$$v(n) = -\sum_{i=1}^{P} a_i x_{n-i}$$

I like to see it this way $x_{n-1} = x(n-i)$
The prediction error becomes

$$y(n) = x(n) - v(n) = \sum_{i=0}^{P} a_i x_{n-i}$$

with $a_0 = 1$

The transfer function reduces to

$$A(z) = \sum_{i=0}^{P} a_i z^{-i} = 1 + a_1 z^{-1} + \ldots + a_P z^{-P}$$

Therefore the problem becomes a minimization problem: find values of a_i that minimize the variance:

$$\begin{aligned}\sigma_Y^2 &= \mathbb{E}[Y(n)] = \mathbb{E}\left[X(n) + \sum_{i=1}^{P} a_i X(n-i)\right]^2 \\ &= \mathbb{E}[X(n)^2 + 2\sum_{i=1}^{P} a_i \mathbb{E}[X(n)X(n-i)] + \sum_{i=1}^{P}\sum_{j=1}^{P} a_i a_j \mathbb{E}[X(n-i)X(n-j)]] \\ &= \sigma_X^2 + 2r^t a + a^t R_X a\end{aligned}$$

where a is the vector with a_i entries and r is the autocorrelation matrix

$$\underline{r} = [r_X(1) \ldots r_X(P)] \quad \mathrm{R}_X = \begin{bmatrix} r_X(0) & r_X(1) & r_X(2) & \ldots & r_X(P-1) \\ r_X(1) & r_X(0) & r_X(1) & \ldots & r_X(P-2) \\ r_X(2) & r_X(1) & r_X(0) & \ldots & r_X(P-3) \\ \ldots & \ldots & \ldots & \ldots & \ldots \\ r_X(P-2) & r_X(P-3) & r_X(P-4) & \ldots & r_X(1) \\ r_X(P-1) & r_X(P-2) & r_X(P-3) & \ldots & r_X(0) \end{bmatrix}$$

$$r_X(k) = \mathrm{E}\{X(n)X(n-k)\}$$

r definition|

by deriving wrt a we get

$$\frac{\partial \sigma_Y^2}{\partial a} = 2r + 2R_x a \rightarrow a_{opt} = -R_X r \rightarrow \sigma_Y^2 = \sigma_X^2 + r^t a_{opt}$$

and the autocorrelation can be estimated as

$$\hat{r}_X(k) = \frac{1}{N} \sum_{n=0}^{N-1-k} X(n)X(n+k)$$

TODO capire meglio

2) Lossless Coding

Lossless coding means to decrease the number of bits needed to encode the data without losing any information: the process is reversible.

The idea is based on variable length coding (VLC) where more probable symbols are encoded in shorter codewords.

Lets start with some notation:

- **Alphabet:** $\mathcal{X} = \{x_1, \dots, x_M\}$ set of symbols to encode
- **Code:** application between $\mathcal{X}$ and $\{0, 1\}$ (set of finite length bit strings)

Digression on fixed length coding (FLC): in FLC all codewords ave the same length, that is M symbols $\rightarrow \lceil \log M \rceil$ bits to encode each symbol.

Example 3 (FLC Text Compression)**.**

An alphabet with 26 symbols is encoded with $\lceil \log 26 \rceil = \lceil 4.7 \rceil = 5$ *bits per codeword. This has a compression ratio of 1.*

In VLC each codeword c_i has length l_i. This is possible if

- **Decodability Condition:** prefix to distinguish codewords
- **Non-equiprobable symbols**

Example 4 (Example on Decodability Condition)**.**

Consider the following alphabet $\mathcal{X} = \{A, B, C\}$ *with the following 4 codes:*

Symbol	Code 1	Code 2	Code 3	Code 4
A	0	0	0	0
B	11	10	01	01
C	111	110	011	11

Coding Schemes

- *Code 1 cannot be decoded uniquely: example 111111 can be either BBB or CC*
- *Code 2 is instaneously decodable (prefix condition)*
- *Code 3 is decodable with delay less than or equal to 1 bit (delay counted from last bit)*
- *Code 4 is decodable with an undefined delay*

2.1) Principle of Information Theory

From the 4 codes before we can see that there is a favorite one, based on the following theorem we can choose to focus on **instantaneous codes**

Theorem 5 (McMillan's Theorem)**.**

Decodable codes do not improve performance with respect to instantaneous codes

Theorem 6 (Kraft's Inequality)**.**

There exists a instantaneous code with lengths $\{l_1, \dots, l_M\}$ *iff*

$$\sum_i 2^{-l_i} \leq 1$$

See proof.

If the equality holds then the code is said to be **optimal**.

Information and Entropy Recap

Suppose symbols $\{x_i\}$ with probabilities p_i.

- The information of a symbol is

$$I(x_i) = -\log_2 p_i = \log_2 \frac{1}{p_i}$$

- The source entropy of $X = \{x_i\}$ is

$$H(X) = -\sum_i p_i \log_2 p_i = -\mathbb{E}[\log_2(p_X(x))]$$

  this is the avg uncertainty of X.
- Joint Entropy:

$$H(X,Y) = -\sum_{i,j} p_{i,j} \log p_{i,j}$$

- Conditional Entropy

$$\begin{aligned} H(X|Y) = \sum_j p_j H(X|Y=y_j) \rightarrow H(X,Y) &= H(Y) + H(X|Y) \\ &= H(Y) + H(Y|X) \end{aligned}$$

- Properties:

$$\begin{aligned} &H(X) > 0 \\ &H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y) \\ &H(X,Y) \leq H(X) + H(Y) \ (= \text{if } \perp) \\ &H(X|Y) \leq H(X) \ (= \text{if } \perp) \\ &H(X) \leq \log_2 M \ (= \text{if } X \sim u) \end{aligned}$$

Lagrange's Method

Lagrange's method is used to solve minmax problems with constraints.

Consider a function $f : x \in \mathbb{R}^n \rightarrow \mathbb{R}$
In order to find the maximum or minimum of f subject to the constraint $\phi(x) = 0$, we look for the stationary points of

$$J(x, \lambda) = f(x) + \lambda\phi(x), \quad \lambda \in \mathbb{R}$$

The stationary points are computed by setting to zero all the derivatives of J:

$$\frac{\partial J}{\partial x_i} = 0, \quad \frac{\partial J}{\partial \lambda} = 0$$

Example 7 (Distribution with Maximum Entropy)**.**

The distribution maximizing the entropy of a M-ary discrete r.v. is found applying the Lagrange's method:

$$p^* = \arg\max_p \sum_{i=1}^{M} p_i \log \frac{1}{p_1}, \qquad \sum_i p_i = 1 \rightarrow \phi(x) = \sum_i p_i - 1 = 0$$

Write J:

$$J(p.\, \lambda) = -\sum_i p_i \log p_i + \lambda \left(\sum_i p_i - 1 \right)$$

Calculate the derivative (specific p_i)

$$\frac{\partial J}{\partial p_i} = -\left(\frac{\log e}{p_i} p_i + \log p_i \right) + \lambda = 0 \rightarrow p_i = \lambda - \log e$$

The max uncertainty is obtained by setting all probabilities equal, that is $p_i = 1/M$.

2.2) Optimal Code

Given a set of M symbols with probabilities p_i we want to find the set of M lengths l_i such that:

- Kraft Inequality is satisfied (prefix code)
- $\forall i = 1, \dots, M,\ l_i \in \mathbb{N}$
- The avg length $L = \sum_i p_i l_i$ is minimized among all sets of l_i
  $\implies$ constrained minimization problem

$$l^* = \arg\min_l \sum_i p_i l_i \quad \text{subject to } \sum_i 2^{-l_i} = 1$$

TODO Proof
The optimal length, which is also the lower bound, corresponds to the entropy:

$$L^* = -\sum_i p_i \log p_i = H(X)$$

However $H(X) \in \mathbb{R}$ not $\mathbb{N}$.
Taking $l_i = \lceil -\log p_i \rceil$ suffices to find a code such that $H(X) \leq L^* < H(X) + 1$

Proof:
Kraft is satisfied:

$$\begin{gathered} l_i = -\log p_i + \delta_i \quad \delta \in [0, 1) \\ 2^{-l_i} = p_i 2^{-\delta_i} = \epsilon_i p_i \quad \epsilon_i \in \left(\frac{1}{2}, 1 \right] \\ \sum_i 2^{-l_i} = \sum_i \epsilon_i p_i \leq \sum_i p_i = 1 \end{gathered}$$

The avg length is:

$$l_i = \lceil -\log p_i \rceil < -\log p_i + 1 \rightarrow p_i l_i = -p_i \log p_i + p_i$$
$$H(X) < \sum_i p_i l_i < \sum_i -pi \log p_i + p_i$$
$$H(X) \leq L^* < H(X) + 1$$

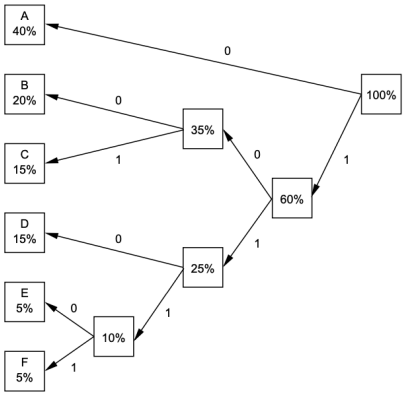
□

Huffman Coding

Huffman discovered how to build the optimal lossless coder for any source with known probabilities.

Algorithm:

1. Create leaf nodes for each symbol, weighted by p_i
2. While there is more than one node:
   - Select two nodes with lowest weights
   - Create a new internal node as their parent
   - Set new node's weight as sum of children's weights
3. Assign '0' to left edges, '1' to right edges
4. Read code for each symbol from root to leaf



Example

This code has avg length $L^* = 2.3$ and entropy $H(X) = 2.246$

As a brief recap consider this:

$$\boxed{\begin{aligned} l_i = \lceil \log_2 p_i \rceil \qquad & H(X) = -\sum_i p_i \log_2 p_i \\ H(X) \leq L^* = \sum_i p_i l_i < H(X) + 1 & \end{aligned}}$$

Block Coding

Instead of mapping one symbol into one codeword, we can map many subsequent symbols into a single codeword. This gives a better performance as:
Let $X^K = (X_1, X_2, \dots X_K)$ a block of K symbols. Then

$$H(X^K) = H(X_1, X_2, \dots, X_K) \leq \sum_{i=1}^{K} H(X_i)$$

Then there are 2 scenarios (L_s is the avg symbol length):

- Symbols not independent:

$$\frac{H(X^K)}{K} < H(X_i) \to L < H(X^K) + 1 \iff L_s < \frac{H(X^K)}{K} + \frac{1}{K}$$

- Symbols identically distributed (but not necessarily independent):

  $$L_s < \frac{H(X^K)}{K} + \frac{1}{K} \leq H(X_i) + \frac{1}{K}$$

  These blocks are encoded via Huffman.

Example 8 (BW image with iid pixels)**.**

Let a BW image have probability p of having a black pixel. Then we group pixels into blocks of 2, the probabilities are:

$$P(BB) = p^2 \quad P(BW) = P(WB) = p(1-p) \approx p \quad P(WW) = (1-p)^2 \approx 1 - 2p$$

from here

$$H(X_1, X_2) = 2H(X) \ll 2$$

Apply the huffman code to the blocks and by an appropriate code we have

$$L \approx 1 \text{ but } L_s \approx \frac{1}{2}$$

Now define the Entropic Rate:

Definition 9 (Entropic Rate)**.**

When the rate is converging, then

$$\mathcal{H}(X) = \lim_{K \to \infty} \frac{H(K)}{K} = \lim_{K \to \infty} L_s^* \leq H(X)$$

Recall the avg length of the optimal code:

$$\begin{gathered} H(X^K) \leq L^* < H(X^K) + 1 \\ \frac{H(X^K)}{K} \leq L_s^* < \frac{H(X^K)}{K} + \frac{1}{K} \\ \downarrow K \to \infty \\ L_s^* \to \mathcal{H}(X) \leq H(X) \end{gathered}$$

where the last steps inequality uses the joint entropy property.

Therefore huffman code is optimal with larger and larger block sizes. This is practically unobtainable as complexity is exponential with K and probability estimation is hard with bigger blocks.

Arithmetic Coding

Arithmetic coding solves the exponential complexity with K increasing. This coding is suboptimal but asymptotically optimal.

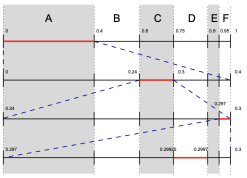
$$L \leq H(X^K) + 2 \to L_s = \frac{L}{K} \overset{K \to \infty}{\longrightarrow} \mathcal{H}(X)$$

This is done achieved by **encoding a sequence as the center interval** with arbitrary precision $q \in [0, 1]$ a fractional number. The arithmetic code can encode blocks of any size, even the entirety of the message.

This requires **2 multiplications and 2 sums per symbol**

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: **ACFD**



- **Start:** $[0, 1)$
- **A:** $[0, 0.4)$
  Center: 0.2. Precision: $p_1/2 = 0.2$.
  $L(1) = -\lceil \log_2 p_1 \rceil + 1$
- **C:** $[0.24, 0.30)$
  Center: 0.275. Precision: $p_1 p_3/2$.
  $L(2) = -\lceil \log_2 p_1 + \log_2 p_3 \rceil + 1$
- **F:** $[0.297, 0.30)$
  Center: 0.2985. Precision: $p_1 p_3 p_6/2$.
  $L(3) = -\lceil \log_2 p_1 + \log_2 p_3 + \log_2 p_6 \rceil + 1$
- **D:** $[0.29925, 0.2997)$
  Center: 0.299475. Precision: $p_1 p_3 p_6 p_4/2$. $L(3) = -\lceil \log_2 p_1 + \log_2 p_3 + \log_2 p_6 + \log_2 p_4 \rceil + 1$

Example

The avg length can be calculated as (iid symbols):

$$
\begin{aligned}
L(n) = -\left\lceil \sum_{i=1}^{n} \log_2 p(x_i) \right\rceil + 1 < -\sum_{i=1}^{n} \log_2 p(x_i) + 2 \\
\overline{L}(n) < \frac{-\sum_{i=1}^{n} \log_2 p(x_i) + 2}{n} \\
L = \mathbb{E}[\overline{L}(n)] < \frac{-\sum_{i=0}^{n} \mathbb{E}[\log_2 p(x_i)] + 2}{n} \\
< \frac{-\sum_{i=0}^{n} H(X_i) + 2}{n} \\
< H(X) + \frac{2}{n} \overset{n \to \infty}{\longrightarrow} H(X)
\end{aligned}
$$

For non iid symbols we have

$$
L(n) < -\log_2 p(x^n) + 2 \rightarrow L = \frac{\mathbb{E}[L(n)]}{n} < \frac{H(X^n) + 2}{n} \rightarrow \mathcal{H}(X)
$$

However $p(x^n)$ is hard to estimate for non iid symbols (if iid $p(x^n) = \prod_i^n p(x_i)$)

Context Based Coding

This approach consists in looking at N_s previous symbols to recognize the context (max $N_c = M^{N_s}$ with M the alphabet) and changes the encoder based on the context. This conditioning reduces the entropy of the source.

2.3) Other Coding Techniques

2.3.1) Exp-Golomb Coding

Universal coding for integer numbers, size (bits) proportional to magnitude

Unisgned Integer

Given int $n \in \mathbb{N}$ the representation consists in

- write $n + 1$ in binary
- use min number of bits: $b = \lfloor \log_2(n + 1) \rfloor + 1$
- place $b - 1$ leading zeroes

Example:

$$
n = 0 \rightarrow \begin{cases} n + 1 = 1_{10} \rightarrow 1_2 \\ b = \lfloor \log_2 1 \rfloor + 1 = 1 \longrightarrow 1 \\ b - 1 = 0 \end{cases}
$$

$$
n = 6 \rightarrow \begin{cases} n + 1 = 7_{10} = 111_2 \rightarrow 1_2 \\ b = \lfloor \log_2 7 \rfloor + 1 = 3 \quad \longrightarrow 00111 \\ b - 1 = 2 \end{cases}
$$

Signed Integer

Given int $n \in \mathbb{Z}$ the representation consists in

- Map $\mathbb{Z} \to \mathbb{N}$ using $m(n) = \begin{cases} 2n-1 & n > 0 \\ -2n & n \leq 0 \end{cases}$
- Use Exp-Golomb for unsigned integer

Example:

$$n = -3 \to m(n) = 6 \to 00111$$

$$n = -6 \to m(n) = 12 \to \begin{cases} n+1 = 13_{10} = 1101_2 \to 1_2 \\ b = \lfloor \log_2 7 \rfloor + 1 = 4 \\ b - 1 = 3 \end{cases} \longrightarrow 0001011$$

2.3.2) Dictionary Based Coding

The idea is to ignore initial statistics and build a dictionary of codewords learned in long term. This allows universal capabilities

Theorem 10 (Asymptotic Optimality Theorem)**.**

For stationary and ergodic sources, the dictionary approach is asymptotically optimal:
With length $n \to \infty$ the average codewrod length L_n converges to the entropy rate $\mathcal{H}$ of the source

Characteristics:

- **Dynamic Dictionary Construction:** Dictionary built on the fly during encoding process
- **Universality:** No prior knowledge of source statistics or probability distributions is required
- **Adaptivity:** adapts to non stationary signals that change over time
- **Practical Efficiency:** used in many formats (zip, gzip, etc)
- **Why:** This approach scales well with long patterns

LZW Encoding (TODO Important non si capisce un cazzo)

It uses a **greedy matching process:**

- Encoder reads symbol by symbol
- For each new input, it checks if the pattern (Prefix W+Next K) already exists
- **Iterative search:** If $W + K$ is found, it continues to read until new pattern is found

Dictionary Evolution and Merging

- **New pattern is found:** Index of longest prefix W is sent to bitstream (output), then $W + K$ is added to dictionary
- **Prefix Merging:** This mechanism merges known prefixes with new suffixes, allowing the dictionary to naturally evolve and "track" local statistics.

EXAMPLE:
Input: 000100000010100001000010
Alphabet: $0, 1$

Initialization: $0 \to 0,\ 1 \to 1$

- First symbol: 0
  The encoder passes through the first symbol: Symbol 0 exists in table (0). Continue search.
  Encoder now sees 00: new symbol, adds it to table: $2 \rightarrow 00$. Write in output the codeword of 0:
  Output = 0
- Second symbol: 0
  We are still at the second 0. 0 is in the table, continue. 00 is in the table (2), continue.
  001 is not in table, add it: $3 \rightarrow 001$
  Output: 0 2
- Fourth symbol: 1
  1 is in table (1), continue. 10 is not in table, add it: $4 \rightarrow 10$
  Output: 0 2 1
- Fifth symbol: 0
  0 is in table, continue. 00 is in table (2), continue. 000 is new, add it to table: $5 \rightarrow 000$
  Output: 0 2 1 2

Here a merge is possible: notice that 2, 3 and 5 have the same prefix 00.

- Seventh symbol. 0
  0 is in table, 00 is in table, 000 is in table (2), 0000 is not in table, add it: $5 \rightarrow 0000$
  Output: 0 2 1 2 2

3) Transform Coding and JPEG

3.1) Block Coding And Quantization

Recall block coding:
Let X_k be a rv with variance σ_k^2, the quantizer distortion is: $D_k = c_k \sigma_k^2 2^{-2R_k}$
Now consider a block of rv (not iid): $X = [X_1, \dots, X_M]^T$

We apply a quantization step:
From the **Huang-Schilteiss formula** the optimal rate per block is:

$$R_k^* = \frac{R_{tot}}{M} + \frac{1}{2}\log\left[\frac{c_k\sigma_k^2}{c_{GM}\sigma_{GM}^2}\right]$$

The optimal case diverges from the uniform Resource distribution case (call $\overline{R} = R_{tot}/M$) we get that the single sample distortion equals to the global distortion:

$$D_k^* = c_{GM}\sigma_{GM}^2 2^{-2\overline{R}}$$
$$\mathcal{D}^* = \frac{1}{M}\sum_{k=1}^{M} D_k = c_{GM}\sigma_{GM}^2 2^{-2\overline{R}}$$

In particular consider the gaussian case (shape factor constant $c_{GM} = x_k = c_{\mathcal{N}}$):

$$\mathcal{D}^* = c_{\mathcal{N}}\sigma_{GM}^2 2^{-2\overline{R}}$$

and if the signals are iid (same variance $\sigma_{GM}^2\sigma_X^2$ and shape factor) the optimal rate is the uniform allocation. For historical reasons it is called PCM:

$$R_k^* = \overline{R} + \frac{1}{2}\log_2 \underbrace{\left[\frac{c_k\sigma_k^2}{c_{GM}\sigma_{GM}^2}\right]}_{=1} = \overline{R}$$
$$\mathcal{D}_{PCM} = c_X\sigma_X^2 2^{-2\overline{R}}$$

Proofs

Using uniform resource allocation the global distortion is:

$$\begin{aligned}\mathcal{D} &= \frac{1}{M}\mathbb{E}[\|X - Q(X)\|^2] = \frac{1}{M}\mathbb{E}[(X-Q(X))^T(X-Q(X))] \\ &= \frac{1}{M}\mathbb{E}[\sum_{k=1}^{M}(X_k - Q(X_k))^2] = \frac{1}{M}\sum_{k=1}^{M}\mathbb{E}[(X_k - Q(X_k))^2] \\ &= \frac{1}{M}\sum_{k=1}^{M} D_k = \frac{1}{M}\sum_{k=1}^{M} c_k\sigma_k^2 2^{-2R_k}\end{aligned}$$

□

The optimal rate is obtained by using the lagrangian multipliers method:
The constraint is: $\sum_{k=1}^{M-1} R_k \leq R_{tot}$
Then the optimization problem becomes:

$$J(R,\lambda) = \frac{1}{M}\sum_{k=1}^{M} c_k\sigma_k^2 2^{-2R_k} + \lambda(\sum_{k=1}^{M-1} R_k - R_{tot})$$

Find the derivatives:

$$\frac{\partial J}{\partial R_k} = -\frac{2\ln 2}{M} c_k \sigma_k^2 2^{-2R_k} + \lambda \qquad \frac{\partial J}{\partial \lambda} = \sum_{k=1}^{M-1} R_k - R_{tot}$$

Set the gradient to 0 and solve:

$$\begin{aligned} -\frac{2\ln 2}{M} c_k \sigma_k^2 2^{-2R_k} + \lambda &= 0 \\ -2R_k &= \log_2(\frac{M\lambda}{2\ln 2}) + \log_2(\frac{1}{c_k\sigma_k^2}) \\ R_k^* &= \underbrace{\frac{1}{2}\log_2\left(\frac{2\ln 2}{M}\right)}_{\lambda'} + \frac{1}{2}\log_2(c_k\sigma_k^2) \end{aligned}$$

Now write it with R_{tot}

$$\begin{aligned} R_{tot} &= \sum_{k=1}^{M} R_k^* = M\lambda' + \frac{1}{2}\sum_{k=1}^{M}\log_2(c_k\sigma_k^2) \\ &\downarrow \\ \lambda' &= \frac{R_{tot}}{M} - \frac{1}{2M}\sum_{k=1}^{M}\log_2(c_k\sigma_k^2) = \frac{R_{tot}}{M} - \frac{1}{2}\log_2\left(\prod_{k=1}^{M}(c_k\sigma_k^2)^{1/M}\right) \\ &= \frac{R_{tot}}{M} - \frac{1}{2}\log_2(c_{GM}\sigma_{GM}^2) \\ &\downarrow \\ R_k^* &= \lambda' + \frac{1}{2}\log_2(c_k\sigma_k^2) = \frac{R_{tot}}{M} - \frac{1}{2}\log_2(c_{GM}\sigma_{GM}^2) + \frac{1}{2}(c_k\sigma_k^2) \\ &= \frac{R_{tot}}{M} + \frac{1}{2}\log_2(\frac{c_k\sigma_k^2}{c_{GM}\sigma_{GM}^2}) \end{aligned}$$

The solution is the **Huang-Schilteiss formula:**

$$R_k^* = \frac{R_{tot}}{M} + \frac{1}{2}\log\left[\frac{c_k\sigma_k^2}{c_{GM}\sigma_{GM}^2}\right]$$

□

Apply the optimal rate to the single distortion

$$D_k^* = c_k\sigma_k^2 2^{-2R_k^*} = c_k\sigma_k^2 2^{-2\overline{R} - \log_2\frac{c_k\sigma_k^2}{c_{GM}\sigma_{GM}^2}} = c_{GM}\sigma_{GM}^2 2^{-2\overline{R}}$$

□

Digression on Arithmetic VS Geometric Mean (AM, GM)

The means are defined as:

$$z_{AM} = \frac{1}{M}\sum_{k=1}^{M} z_k \geq z_{GM} = \sqrt[M]{\prod_{k=1}^{M} z_k}$$

Proof

By Jenses Inequality we have:

$$f(\frac{1}{M}\sum_{k=1}^{M} z_k) \geq \frac{1}{M}\sum_{k=1}^{M} f(z_k)$$

with f the log function.

$$\log(z_{AM}) \geq \frac{1}{M}\sum_{k=1}^{M}\log(z_k) = \log\left(\prod_{k=1}^{M} z_k^{1/M}\right) = \log(z_{GM}) \to z_{AM} \geq z_{GM}$$

3.2) Transform Coding

The aim is to make the signal sparse, that is, few large samples (low rate EQ) and many small ones (high EQ). This transforms the id (not iid) signal into blocks with diverse variances (same mean) so to minimize σ^2_{GM}.
For example a fourier transform of a pure sound is sparse: All zereos except for f_w.

The operator has the following properties:

- Reversible: $Y = T(X) \iff X = T^{-1}(Y)$
- Input in $\mathbb{R}^M$: an image is $\mathbb{R}^2$
- Input is not sparse
- Output is sparse

We consider $Y = \mathcal{T}X$ where $\mathcal{T}$ is an invertible matrix

- Inverse exists by definition
- This acts as a basis change: basis is set of signals to reconstruct intended signal
- If $\mathcal{T}$ is orthogonal the quantization is the same

So the paradigm becomes:

$$x \to y = \mathcal{T}x \to \hat{y} = Q(y) \to \hat{x} = \mathcal{T}^{-1}\hat{y}$$

Orthogonal Transform

Orthogonal Transform has $\mathcal{T}$ as an orthogonal matrix. This gives many advantages:

- Inverse is immediate: $\mathcal{T}^{-1} = \mathcal{T}^T$
- Is an isometry keeps $\mathcal{L}^2$ norm of any X: $\|\mathcal{T}X\|^2 = \|X\|^2$
- Isometry keeps distortion the same on the output: $D_X = D_Y$

Now consider block coding on id (not iid) gaussian rvs. With orthogonal transform we get:

- Distortion can be applied directly to the quantized output: $D_T = D_Y \overset{\text{optimal } R_k}{=} c_{GM,Y}\sigma^2_{GM,Y}2^{-2R_k}$
- Variance of Y is the AM of X: $\sigma^2_{AM,Y} = \sigma^2_X$
- AM variance is constant as it is the energy of the signal ($\mathcal{L}^2$ norm unchanged)

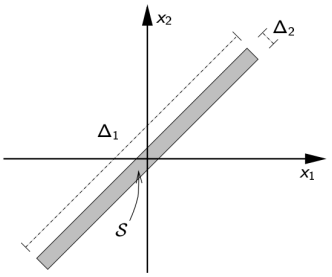
Finally define the **Coding Gain:**

$$G_T = \frac{D_{PCM}}{D_T} = \frac{c_{\mathcal{N}}\sigma^2_X 2^{-2R_k}}{c_{\mathcal{N}}\sigma^2_{GM,Y}2^{-2R_k}} = \frac{\sigma^2_{AM,Y}}{\sigma^2_{GM,Y}} \geq 1$$

this shows that if the transform reduces the GM variance (with non id samples), then the transform is more efficient.

Example.

Consider a signal where $X_1, X_2 \sim u\left[-\frac{\Delta_1}{2\sqrt{2}}, \frac{\Delta_1}{2\sqrt{2}}\right]$ *and have the joitn distribution* $f_{X_1,X_2}(x_1,x_2) = \begin{cases} \frac{1}{\Delta_1\Delta_2} & (x_1,x_2) \in S \\ 0 & (x_1,x_2) \notin S \end{cases}$ *there* S *is an oriented rectangle* (45°) *with dimension* $\Delta_1 \times \Delta_2$. *With* $\Delta_1 \gg \Delta_2$.



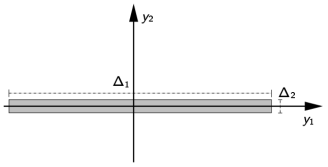
Image

Apply uniform quantization (best fit for uniform rv, but not optimal) directly on the samples:
Notice that X_1, X_2 have the same variance $\sigma^2 = \Delta_1^2/24$ and form factor $c = 1$
Based on how many bits are assigned to the single rv we get

bits	R	D_1	D_2	D
0	0	σ^2	σ^2	$2\sigma^2$
1	0.5	$\sigma^2/4$	σ^2	$\frac{5}{4}\sigma^2$
1	0.5	σ^2	$\sigma^2/4$	$\frac{5}{4}\sigma^2$
2	1	$\sigma^2/4$	$\sigma^2/4$	$\frac{1}{2}\sigma^2$
2	1	$\sigma^2/16$	σ^2	$\frac{17}{16}\sigma^2$
3	1.5	$\sigma^2/16$	$\sigma^2/4$	$\frac{5}{16}\sigma^2$
4	2	$\sigma^2/16$	$\sigma^2/16$	$\frac{1}{8}\sigma^2$

Table

Apply the following transform $\mathcal{T} = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & -1\\ 1 & 1\end{bmatrix}$ (orthogonal since $\mathcal{T}^T\mathcal{T} = I$) then the rvs become: $Y_i \sim u\left[-\frac{\Delta_i}{2}, \frac{\Delta_i}{2}\right]$ and the joint distribution $f_{Y_1,Y_2}(y_1, y_2) = \begin{cases}\frac{1}{\Delta_1\Delta_2} & (y_1, y_2) \in S'\\ 0 & (y_1, y_2) \notin S'\end{cases}$ where S' is the rotated surface from before



Image

Now the variances are different $\sigma_1^2 = \Delta_1^2/12 = 2\sigma^2,\ \sigma_2^2 = \Delta_2^2/12 \ll \sigma^2$ and the distortion becomes:

bits	R	D_1	D_2	D
0	0	$2\sigma^2$	$\ll \sigma^2$	$2\sigma^2$
1	0.5	$\sigma^2/2$	$\ll \sigma^2$	$\frac{1}{2}\sigma^2$
2	1	$\sigma^2/8$	$\ll \sigma^2$	$\frac{1}{8}\sigma^2$
3	1.5	$\sigma^2/32$	$\ll \sigma^2$	$\frac{1}{32}\sigma^2$

Table

PROOFS

Inverse is by definition.

□

Isometry:

$$\|\mathcal{T}X\|^2 = (\mathcal{T}X)^T(\mathcal{T}X) = (X^T\mathcal{T}^T)(\mathcal{T}X)X^T(\mathcal{T}^T\mathcal{T})X = X^TX = \|X\|^2$$

And distortion on isometry:

$$D_Y = \frac{1}{M}\mathbb{E}[\|Y-\hat{Y}\|^2] = \frac{1}{M}\mathbb{E}[\|\mathcal{T}(X-\hat{X})\|^2] = \frac{1}{M}\mathbb{E}[\|(X-\hat{X})\|^2] = D_X$$

□

Same distortion:

$$\mathcal{D}_T = \frac{1}{M}\mathbb{E}[\|X-\hat{X}\|^2] = \frac{1}{M}\mathbb{E}[\|\mathcal{T}^{-1}Y-\mathcal{T}^{-1}\hat{Y}\|^] = \frac{1}{M}\mathbb{E}[\|Y-\hat{Y}\|^2] = D_Y$$

□

Variance equivalences:

$$\begin{aligned}\sigma_X^2 = \sigma_{AM,X}^2 &= \frac{1}{M}\sum_{k=1}^{M}\mathbb{E}[X_k^2] = \frac{1}{M}\mathbb{E}\left[\sum_{k=1}^{M}X_k^2\right] = \frac{1}{M}\mathbb{E}[\|X\|^2] \\ &\quad \text{(recall: } \|Y\|^2 = \|\mathcal{T}X\|^2 = \|X\|^2\text{)} \\ &= \frac{1}{M}\mathbb{E}[\|Y\|^2] = \frac{1}{M}\mathbb{E}\left[\sum_{k=1}^{M}Y_k^2\right] = \frac{1}{M}\sum_{k=1}^{M}\mathbb{E}[Y_k^2] \\ &= \sigma_{AM,Y}^2\end{aligned}$$

□

3.3) Practical Implementation of Huang-Schulteiss

HS has a series of limitations:

- Returns negative values
- Returns non-integer values

The **Modified HS algorithm** is very simple:

1. Use HS with all M components
2. If there are negative values, set $R = 0$ of the component and recalculate HS with $M - N$ components
3. When there are only positive values, floor the results
4. Calculate residual rate and allocate eventual residual bits to the results that got rounded the most

Example.

Let there be a gaussian input with 4 components and $R_{tot} = 10$:

$$\sigma_1^2 = 1000,\ \sigma_2^2 = 100,\ \sigma_3^2 = 50,\ \sigma_4^2 = 1$$

The GM is ≈ 47.29
We get:

$$R(1) \approx 4.7,\ R(2) \approx 3.04,\ R(3) \approx 2.54,\ R(4) \approx -0.28 \overset{\text{negative}}{\longrightarrow} 0$$

Recompute HS with 1,2,3:
GM ≈ 171
We get (also floor):

$$R(1) \approx 4.61 \to 4,\ R(2) \approx 2.95 \to 2,\ R(3) \approx 2.45 \to 2,\ R(4) = 0 \to 0$$

Since $4+2+2+0 \neq 10$ *we can allocate 2 residual bits to components 1 and 4*

$$R(1) = 5,\ R(2) = 3,\ R(3) = 2,\ R(4) = 0$$

The total distortion is:

$$D = \sum D_i = 0.98 + 1.56 + 3.13 + 1 = 6.67$$

The **greedy algorithm** returns the same allocation but is faster:

- Initialization. $R_k = 0\ \forall k \in \{0, \dots, M-1\} \qquad D_k = \sigma_k^2 \forall k \in \{0, \dots, M-1\}$
- While $\sum R_k \leq R_{tot}$
  - $l = \arg\max_k D_k$
  - $R_l \leftarrow R_l + 1$
  - $D_l \leftarrow D_l / 4$
    This takes R_{tot} iterations

Example.

Consider the same data as before:

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
0	-	0	0	0	0	1000.00	100.00	50.00	1.00
1	1	1	0	0	0	250.00	100.00	50.00	1.00
2	1	2	0	0	0	62.50	100.00	50.00	1.00
3	2	2	1	0	0	62.50	25.00	50.00	1.00
4	1	3	1	0	0	15.63	25.00	50.00	1.00
5	3	3	1	1	0	15.63	25.00	12.50	1.00

Iterations 1-5

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
6	2	3	2	1	0	15.63	6.25	12.50	1.00
7	1	4	2	1	0	3.91	6.25	12.50	1.00
8	3	4	2	2	0	3.91	6.25	3.13	1.00
9	2	4	3	2	0	3.91	1.56	3.13	1.00
10	1	5	3	2	0	0.98	1.56	3.13	1.00

Iterations 6-10

3.4) Orthogonal Transforms For Compression

Now let's find out how to build the orthogonal transform matrices for:

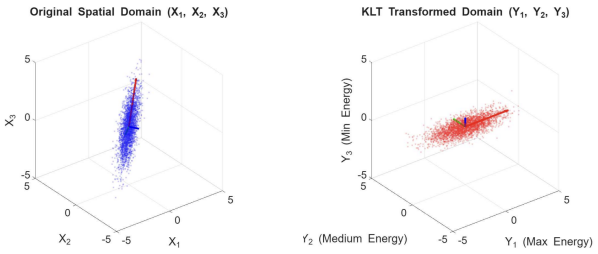
- A rv with known statistical properties
- The general case

Karhunen-Loeve Transform (KLT)

Let X be a 0 mean random vector of size M and correlation $R_X = \mathbb{E}[XX^T]$. With most relevant data we can consider that R_X has M eigenvectors $u_1, \dots, u_M$. The KLT (or principal component analysis (PCA) in ML) matrix has for rows the eigenvectors

$$\mathcal{T}_{KLT} = [u_1, \dots, u_M]^T$$

This is aligning the axis to the direction in which the data has the max variance



Graphical Example

It has the following properties:

- Orthogonal: $\mathcal{T}_{KLT}^{-1} = \mathcal{T}_{KLT}^{T}$
- It is decorrelating: $\mathbb{E}[Y_i Y_j] = \lambda_i \delta_{ij}$
- Best energy concentration (sparsity): $\sum \mathbb{E}[Y_i] \geq \sum \mathbb{E}[(T_i X_i)^2]$ where T_i is a generic orthogonal matrix
- Optimal for gaussian vectors: $\sigma^2_{GM,\mathcal{N}} \leq \sigma^2_{GM}$
- Among all linear orthogonal transforms, the KLT is optimal for energy compaction: for any $N < M$, the sum of the first N variances is maximized by the KLT.

However this approach has many downsides:

- KLT is data dependent: statistics must be known and are hard to estimate, they also change for each dataset
- Computationally expensive: $O(N^3)$ for eigenvector computation and $O(N^2)$ for multiplication
- Since data depends, the basis function (matrix) must also be sent with the data
- Assuming stationarity is not always true

For a Markov Process (AR(1)) with correlation $\rho \to 1$, frequency transforms (DFT, DCT) offer near-optimal performance with fixed basis functions and fast algorithms ($O(NlogN)$).

Discrete Fourier Transform (DFT)

The 1D case is the following:

$$y[k] = \frac{1}{\sqrt{M}} \sum_{n=1}^{M} x[n] e^{-j\frac{2\pi}{M}kn}$$

Or in matrix form:

$$\mathcal{T}_{DFT} = \frac{1}{\sqrt{M}} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & W_M & W_M^2 & \dots & W_M^{M-1} \\ 1 & W_M^2 & W_M^4 & \dots & W_M^{2(M-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & W_M^{M-1} & W_M^{2(M-1)} & \dots & W_M^{(M-1)(M-1)} \end{bmatrix}$$

Where $W_M = e^{-j2\pi/M}$ is the M-th primitive root of unity
Each row is therefore the conjugate of a basis vector.
the total energy is preserved: $\|y\|^2 = \|x\|^2$

In 2D, the DFT is a separable transform:

$$Y = \mathcal{T} X \mathcal{T}^T$$

this computes the rows and then the columns (horizontal and vertical frequency analysis)

This transform decomposes the image into a weighted sum of N^2 orthogonal basis patterns:

$$B_{k,l}(n,m) = \frac{1}{N} e^{j\frac{2\pi}{N}(kn+lm)}$$

each coefficient $Y[k,l]$ represents the frequency component of horizontal and vertical frequency combination
Most energy is concentrated in the low frequencies.

This is still not ideal since the DFT suposes a periodic signal. We compress finite signals and therefore on the image edges we have spectral leakage (low frequencies leak into high ones) making the signal less sparse

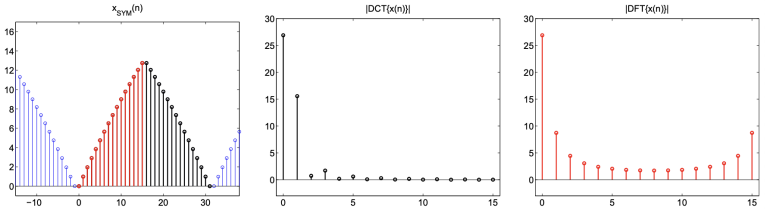
Discrete Cosine Transform (DCT)

A general approach is used **Discrete Cosine Transform (DCT)**
Each entry follows the form:

$$(\mathcal{T}_{DCT})_{k,n} = \begin{cases} \frac{1}{\sqrt{N}} & k = 0 \\ \sqrt{\frac{2}{N}} \cos(\frac{(2n+1)k\pi}{2N}) & k > 0 \end{cases}$$

DFT is not used since DFT has high frequency components near the signal edges. The DCT is a way to mirror the signal before the periodicity. It has only positive frequencies

Applying the DCT to a signal (a sequence of N real numbers) produces N real coefficients and has a better sparsification property than DFT thanks to the symmetric periodization.



Example With Mirroring

The DCT is also separable:

$$Y = \mathcal{T}_{DCT} X \mathcal{T}_{DCT}^T$$

A large-size, non-stationary image is more conveniently represented by dividing it into small blocks.

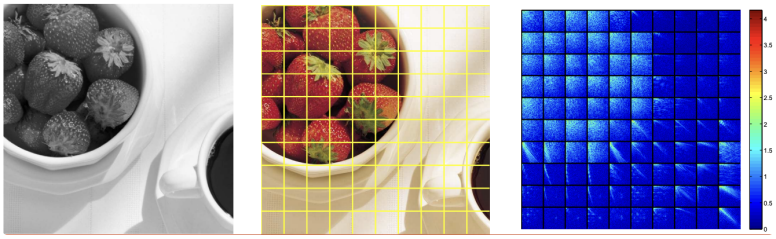
Each value follows the form

$$C(i,j) = \frac{1}{4}\alpha(i)\alpha(j)\sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y)\cos\left[\frac{(2x+1)i\pi}{16}\right]\cos\left[\frac{(2y+1)j\pi}{16}\right]$$

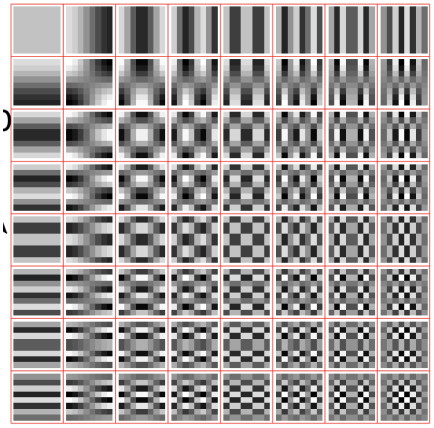
with α a normalization factor $\alpha(u) = \begin{cases} \frac{1}{\sqrt{2}} & \text{if } u = 0 \\ 1 & \text{if } u > 0 \end{cases}$.

Reason on the DC component: $C(0,0) = \frac{1}{4}\frac{1}{2}\sum\sum f(x,y)\cdot 1\cdot 1 = \frac{1}{8}\sum\sum f(x,y)$
After centering the signal, the dc component is $\in [-1024, 1016]$.

Example: 8 × 8 block-based DCT. Each 8 × 8 block of pixels from the image is projected onto the 64 basis vectors:
The corresponding scalar product is the DCT coefficient telling how much the block is similar to the basis vector



Block DCT



8x8 Basis

The sparsification allows to give higher bits to many small valued coefficients and lower bits to less frequent bigger values. How are the quantization coefficients computed?

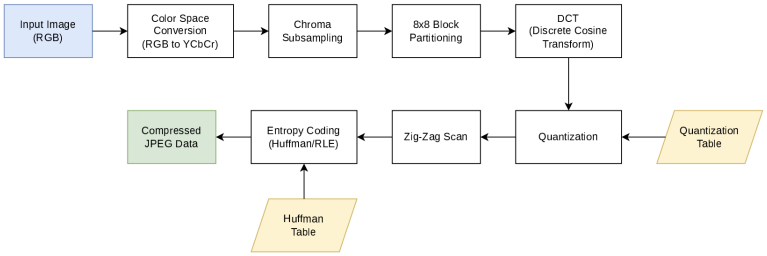
- HS formula (practical implementation)
- Fixed steps (JPEG)

Take a 8×8 block:

- Center the signal (subtract 128)
- Calculate the DCT coefficients
- This returns 64 coefficients (can be used to reconstruct og signal)
- The value at $(0, 0)$ is the mean intensity called **DC** value
- The other values are called AC values

3.5) JPEG Standard (TODO)

JPEG is an image compression standard defined in 1991 that defines **only the decoder** for interoperability and implementation competition



JPEG Scheme

The steps are the following: pre processing: color space transform, chrome subsampling, block split and average removal $\rightarrow$ DCT $\rightarrow$ Quantize $\rightarrow$ VCL

DCT is performed on 8×8 blocks (small improves stationarity, large correlation, 8 is a good mittleground)

The quantization is defined as uniform quantization (mid-thread):

$$c_{ij} = \text{round}(\frac{c_{ij}}{q_{ij}})$$

but q is not defined by standard and must be encoded

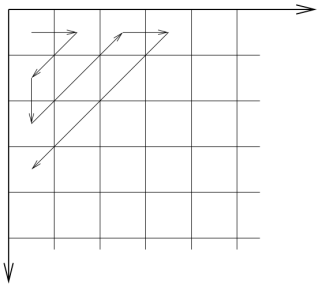
A quantzation parameter is the **quality factor** $Q \in [1, 100]$ that controls the scaling factor:

$$S_F = \begin{cases} \frac{5000}{Q} & 1 \leq Q \leq 50 \\ 200 - 2Q & 50 < Q \leq 99 \\ 1 & Q = 100 \end{cases} \rightarrow q \leftarrow \frac{S_F}{100} q$$

A zig-zag scan is performed on the quantized values in order to encode them in a single string, where

- the first value is the Difference of the DC component of this block and the previous block
- the next values are a pair of numbers representing (# of zeroes in scan, value of first non zero)
- final EOB special symbol is added to end the string it is $(0, 0)$

$DC_n - DC_{n-1}$	(# of zeroes, non zero coeff value)	...	**EOB** $(0, 0)$



Zig-Zag scan

61	16	3	0	0	0	0	0
3	3	0	-1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

61-DC_{n-1}	0,16	0,3	1,3	0,3	7,-1	EOB

Example of encoding

Encoding

The symbols are then encoded:
DC Encoding:

JPEG encodes the DC difference with a pseudo-huffman code.
There are $k \in \{0, \dots, 11\}$ categories, each of them holding 2^k values using 2's complement. Each

category is assigned a codeword (see table).

Call DC_p the DC difference, the category is chosen $k = \lceil \log_2(|DC_P| + 1) \rceil$
To the category the binary value of DC_P is added as a suffix. If it is negative each value is complemented.

Category	Code
0	00
1	010
2	011
3	100
4	101
5	110
6	1110
7	11110
8	111110
9	1111110
10	11111110
11	111111110

Category Code

For example suppose $DC_P = -5_{10} = 101_2$ that must be complemented to 010. The category is $k = 3 \rightarrow 100$ and thus the codeword is 100 010.

DC values have a range $\in [-1024, 1060]$ a difference of two DC signals is $\in [-2040, 2040]$ and thus with cardinality 4081. Since there are 11 categories we have $\sum_{k=0}^{11} 2^k = 2^{12} = 1096 > 4081$ values.

AC Encoding
Recall that the AC coefficients are previously encoded using (a, b) with a the # of zeroes before b the first non-zero AC value.

- b is encoded as with 2's complement.
- the prefix code is specified by (R, C) (Run, category). Run $= a \in [0, 15]$ while Level is the class(k) of b. These are saved in a table.

Example: Encode (3,16):

- Encode 16: $k = 5$ and $16_{10} = 10000_2$
- $(L, C) = (0, 5) \overset{\text{table}}{\rightarrow} 11010$
  The codeword is 11010 10000

Two custom codewords are described:

- End Of Block (EOB) $= (0, 0) \rightarrow 1010$
- Zero Run (ZR) $= (15, 0) \rightarrow 11111111001$

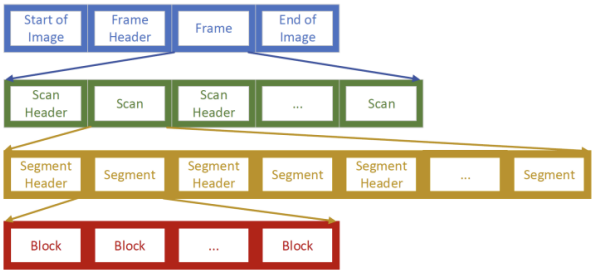
Recap the steps:

- Take 8x8 block
- Center it: subtract 128
- Calculate DCT coefficients and obtain matrix of 64 coefficients $c_{i,j}$. We call $c_{0,0}$ the DC component (mean luminosity) and the others AC components
- Take a quantization matrix with entries $q_{i,j}$ scaled by the scaling factor.
- Quantize the values of the DCT using a mid-thread quantization $c_{ij} = \text{round}(\frac{c_{ij}}{q_{ij}})$

- Using the zig-zag scan, build the string
- Encode the DC difference and the (a, b) values

Frame Building

The standard frame follows this logic:



Frame

- Frame header contains static info (size, color space, digitalization format)
- Image is stored in a frame as various scans
- Scan header contains quantization table (luminance and chrominance)
- A single scan contains various segments, each segment is a concatenation of blocks
- Segment header contains huffman tables

JFIF (JPEG File Interchange Format) is the standard format for metadata in JPEG files

4) Wavelet

4.1) Discrete Wavelet Transform

Recall the principle of a spectrum analyzer (short time fourier transform) (DSP 2). Frequency and time resolutions are inversely proportional to each other

$$\text{Heisenberg-like Uncertainty principle: } \Delta t \cdot \Delta f \geq \frac{1}{4\pi}$$

Wavelet is the tool that allows the block of the JPEG to scale dynamically based on frequency (high frequency, smaller blocks. Low frequency, large blocks). In fact an image is made of two parts:

- Anomalies: High frequency content (edges, contours). This needs a good time resolution to see where they are located
- Trends: low frequency content (smooth areas, textures). This needs good frequency resolution to better capture subtle shifts in the image

To achieve this we use a **mother wavelet** $\psi(t)$ and generate the basis through scaling and translation:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}}\psi\left(\frac{t-b}{a}\right)$$

This works with the

Theorem 11 (Universal Principle)**.**

The linear transforms used in signal processing and compression are defined by projection of the input signal onto an appropriate set of basis functions.

Given an orthonormal basis $\{\phi_k(t)\}$ *any signal can be perfectly represented as*

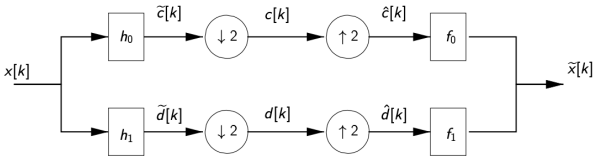
$$x(t) = \sum_k c_k \phi_k(t)$$

and the coefficient is obtained by

$$c_k =< x(t), \phi_k(t) >= \int x(t)\phi_k^*(t)dt$$

Filter Bank

The idea is to divide the signal in two parts: high and low frequency. These will have their bandwidth halved and so they get decimated and interpolated with a factor of 2. These get recombined to get a delayed copy of the original signal.



Filter Bank Scheme

Perfect Reconstruction (PR)

Let $x[k]$ be the original signal and $\tilde{x}[k]$ the signal after passing through the filter bank. Perfect reconstruction is achieved if $\tilde{x}[k]$ is a delayed copy of $x[k]$, that is:

$$\tilde{x}_k = x_{k+l} \iff \tilde{X}(z) = z^{-l}X(z)$$

It turns out that in the Z domain the output is:

$$\begin{aligned}\tilde{X}(z) &= \frac{1}{2}[F_0(z)H_0(z) + F_1(z)H_1(z)]X(z) \\ &+ \frac{1}{2}[F_0(z)H_0(-z) + F_1(z)H_1(-z)]X(-z) \\ &= T(z)X(z) + A(z)X(-z)\end{aligned}$$

$$\tilde{X}(z) = \frac{1}{2}\underbrace{\begin{bmatrix} H_0(z) & H_1(z) \\ H_0(-z) & H_1(-z) \end{bmatrix}}_{\text{modulation matrix}} \cdot \begin{bmatrix} F_0(z) \\ F_1(z) \end{bmatrix} X(z) = \frac{1}{2}\begin{bmatrix} 2z^{-l} \\ 0 \end{bmatrix} X(z) = z^{-l}X(z)$$

Where we set the following bounds to obtain PR:

- $T(z) = 2z^{-l}$ is the Non Distortion (ND) condition
- $A(z) = 0$ the Aliasing Cancelation (AC) condition

The modulation matrix needs to be invertible:

$$\forall z \in \mathbb{C} : |z| = 1,\ \Delta(z) = H_0(z)H_1(-z) - H_1(z)H_0(-z) \neq 0$$

There are 2 types of Filters that can be used:

- **Quadrature Mirror Filters:**

  $$H_0(z) = H_1(-z) \text{ and } F_0(z) = H_0(z),\ F_1(z) = -H_1(z)$$

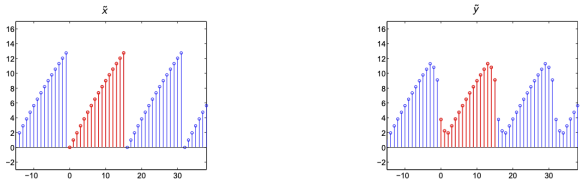
- **Conjugate Quadrature Filters:**

  $$H_0(z) = H_1(-z) \text{ and } F_0(z) = H_0(z^{-1}),\ F_1(z) = -H_1(z^{-1})$$

  Both are Orthogonal and energy conserving. A special case is the **Haar filter**, which is both A QMF and CQF at the same time (see later).

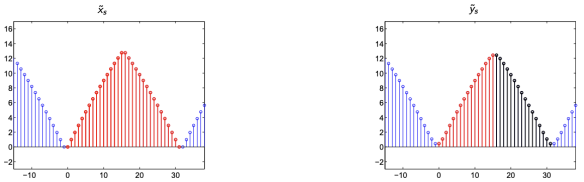
All this works only for infinite length signals. What happens when signals are finite?

- **Standard approach:** zero padding and DTFT. This produces **coefficient expansion,** the output signal has $N + M - 1$ coefficients (N input size and M filter size).
- **Circular Convolution:** This is obtained via periodicization of the signal. This however introduces boundary artifacts (aliasing in frequency) because of the implicit periodicization of the circular convolution (DFT)



Example Of Boundary Artifacts

- **Symmetrization:** We create a new signal by adding a mirror image of the original signal to the period. Let x have period N then x_s has period $2N$. The circular convolution (with periodic filter) will return a periodic and symmetrical signal and thus only the first N samples have to be computed. This does not create artifacts.



Example of Symmetric signal (in black the N not calculated coefficients)

Haar and Biorthogonal Filters

The only symmetric FIR orthogonal filter is the Haar filter

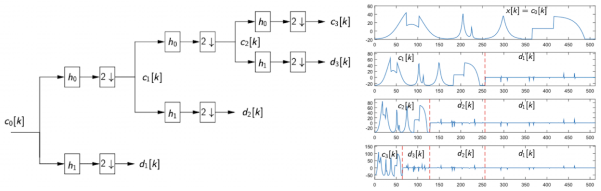
$$h_0[k] = [1, 1] \quad f_0[k] = [1, 1]$$
$$h_1[k] = [1, -1] \quad f_1[k] = [-1, 1]$$

Unfortunately this is a filter with Vanishing moment (VM) of $p = 1$. The high pass filter will not respond to polynomials with degree $< p$. We need at least $2p$ taps.

Haar has $p = 1$ and can only represent piecewise linear functions.
We consider the **Cohen-Daubechies-Fauveau (CDF)** biorthogonal filter:

- Symmetric
- Maximize VM
- close to orthogonal ($\omega_i \approx 1$)

They work by decomposing the signal into three wavelets

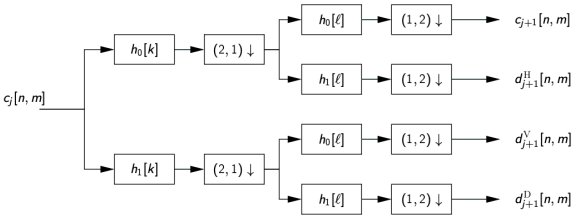


Three wavelet decomposition

and then reconstructing the signal in order.

2D Wavelet Decomposition

For 2D signals it is possible to use this schema:

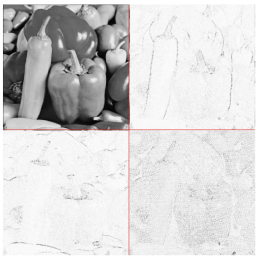


2D

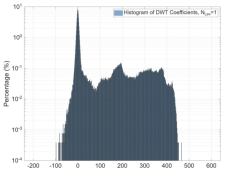
and for multi level decomposition just the output c_i is used.
The 4 outpust are:

- c (A): approximation coefficients (low res version of og image by LP filter in both directions)
- d^H (H): horizontal HP (HP on rows, LP on cols)
- d^V (V): vertical HP (HP on cols, LP on rows)
- d^D: diagonal details

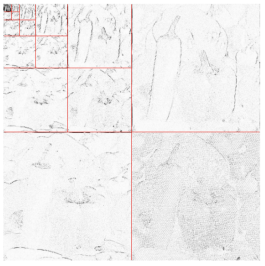
Applying this sparsifies the signal:



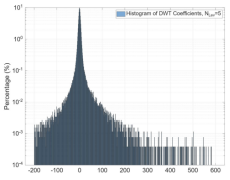
After one level of 2D-DWT, we achieve some sparsity, but still 1/4 of coeff's (LL subband) are relatively large.



One Level



This trend continues with 3 to 5 levels of decomposition.



5 Levels

the optimal levels are between 4-6.

Proof of output equation:
Analyze the bank in the Z domain:
We first pass through the filter:

$$\tilde{c}[k] = (h_0 * x)(k) \xrightarrow{\mathcal{Z}} \tilde{C}(z) = \sum_n \tilde{c}_n z^{-n} = H_0(z)X(z)$$

Then decimate:

$$c[k] = \tilde{c}[2k] \xrightarrow{\mathcal{Z}} C(z) = \frac{1}{2}\left[\tilde{C}(z^{1/2}) + \tilde{C}(-z^{1/2})\right]$$

Then Interpolate:

$$\hat{c}[k] = \begin{cases} c[k/2] & k \text{ even} \\ 0 & k \text{ odd} \end{cases} \xrightarrow{\mathcal{Z}} \hat{C}(z) = C(z^2)$$

The same is done for $d \to \hat{D}(z) = D(z^2)$. And the output is the sum of these signals through the filters:

$$\tilde{x}[k] = (f_0 * \hat{c})(k) + (f_1 * \hat{d})(k) \xrightarrow{\mathcal{Z}} F_0(z)C(z^2) + F_1(z)D(z^2)$$

Written in terms of the input signal:

$$\begin{aligned}\tilde{X}(z) = &\frac{1}{2}[F_0(z)H_0(z) + F_1(z)H_1(z)]X(z) \\ &+ \frac{1}{2}[F_0(z)H_0(-z) + F_1(z)H_1(-z)]X(-z)\end{aligned}$$

□

Proof of Interpolation and Decimation:
To interpolate we notice that the output becomes

$$\hat{c}[k] = [c_0, c_1, c_2, \ldots] \to c[n] = [c_0, 0, c_1, 0, c_2, 0, \ldots]$$

Since every second element is zero, we define the z transform with a index substitution $n = 2k$

$$C(z) = \sum_n c[n] z^{-n} = \sum_k c[2k] z^{-2k} = \sum_k \hat{c}[k] z^{-2k} = \hat{C}(z^2)$$

With this in mind the decimation is similar:
To compute it we first define a signal $s[k]$ that is the interpolated signal of $\tilde{c}[k]$. From the earlier result we know that $S(z) = \tilde{C}(z^2) \to \tilde{C}(z) = S(z^{1/2})$
Therefore we just need to compute the z transform of $s[k]$

$$s[n] = \frac{1 + (-1)^n}{2} \tilde{c}[n] = \begin{cases} \tilde{c}[n] & n \text{ even} \\ 0 & n \text{ odd} \end{cases} = [\tilde{c}[0], 0, \tilde{c}[2], 0, \tilde{c}[4], 0, \ldots]$$

Then take the Z transform of s

$$\begin{aligned} S(z) &= \sum_n \left(\tilde{c}[n] \cdot \frac{1 + (-1)^n}{2} \right) z^{-n} \\ &= \frac{1}{2} \sum_n \tilde{c}[n] z^{-n} + \frac{1}{2} \sum_n \tilde{c}[n] ((-1)z)^{-n} \\ &= \frac{1}{2} \left[\tilde{C}(z) + \tilde{C}(-z) \right] \\ \tilde{C}(z) = S(z^{1/2}) &= \frac{1}{2} \left[\tilde{C}(z^{1/2}) + \tilde{C}(-z^{1/2}) \right] \end{aligned}$$

□

4.2) Image Compression with Wavelets (TODO)

Embedded Zerotrees of Wavelet Coefficients (EZW)

5) Learned Image Coding

This is the state of the art of compression techniques:

Definition 12 (Learned Image Compression (LIC))**.**

Learned Image Compression (LIC) or Neural Image Compression (NIC) refers yo the application of neural networks and machine learning to data compression tasks

This is the state of the art since they don't use hand crafted rules but LIC algorithms are learned and outperform traditional algorithms.

- Classic: It uses linear transforms to de-correlate pixels based on statistical assumptions
- Neural: Learned non-linear transforms optimized directly for the data

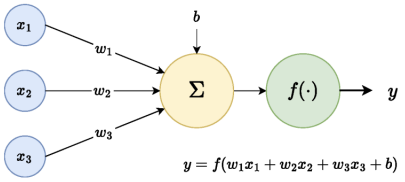
NN can be used twofold:

- **Combinatorial Selection (Classic):** hand crafted tools and ai optimizes their parameters
- **Continuous Learning (Neural):** the filter is found via gradient descent

DL Basics

Recall how the perceptron works:

$$y = f(w^T x + b) = f\left(\sum_i w_i x_i + b\right)$$



Example

FFNN

Multi-Layer Perceptron (FFNN)

The output of a neuron is based on all the outputs of previous layer.

- The input layer gives as output the input
- The hidden layers give as output the output of their perceptrons
- The output layer is same as hidden layer but with different activation function

The learning follows the gradient descent rule:

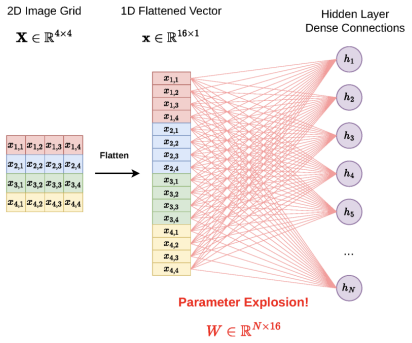
$$\theta^{(i+1)} = \theta^{(i)} - \eta \nabla_\theta \mathcal{L}(\theta^{(i)})$$

which can be easily computed by backpropagation.

This is not good for images:

- High dimension/Parameter Explosion: only 1D vector, so images are straightened and spatial correlations are lost

- No translation invariance
- Training difficulty



Example

CNN

Convolutional Neural Networks (CNN)
Instead of relying on all the previous outputs, CNNs use a subset of outputs but **share the same weight**.
This allows for:

- Local connectivity and weight sharing: this brings to translation invariance and reduces parameter count
- Analysis: Convolutions compact information
- Synthesis: Transposed convolutions reconstruct the full-resolution image

however, multiple filters are needed as each filter learns a specific detector (edges, textures, ...). Multiple kernels work in parallel and therefore the 2D input becomes a 3D tensor

Activation Function

The activation function is different than the standard ones (relo, sigmoid, ...). Instead for compression the **Generalized Divisive Normalization (DGN):**

$$w_i = \frac{v_i}{\sqrt{\beta_i + \sum_j \gamma_{ij} v_j^2}}$$

where β_i and γ_{ij} are learned parameters.
This allows for:

- Lateral inhibition: the denominator measures local energy, if neighbors are big, this feature is suppressed
- This works similar to the human visual system
- decorrelates the signal making the distribution roughly normal

Neural Paradigm

Instead of transforming fixed blocks, neural compression transforms the entire image into a deep 3D Latent Tensor
As explained before, this allows for spatial reduction and channel expansion (parallel features extraction)

The traditional encoding is replaced by an autoencoder:

- Analysis Transform (g_a) (encoder): Uses CNN to map image x to the tensor y
- Bottleneck (compression): latents y are quantized to $\hat{y}$ to be entropy encoded
- Synthesis transform (g_s) (decoder): learns to reconstruct image from the quantized latents $\hat{y}$

The optimization objective now becomes the minimization of the lagrangian:

$$\mathcal{L} = R + \lambda D = \underbrace{D_{KL}[q(\hat{y}|x)\|p(\hat{y})]}_{\text{Rate}} + \underbrace{\lambda \mathbb{E}[\rho(x,\hat{x})]}_{\text{Distortion}}$$

In Neural Compression, instead of compressing the image, the output of the Analysis Transform ($y = g_a(x)$) is quantized.

Pixel Space	Latent Space
High Redundancy	Features are decorrelated
Probabilities are hard to model	More predictable distributions (gaussian/Laplace)
Visible quantization errors (blocking, ringing)	Network hides noise in less perceptually important channels

There is a problem with gradient descent in the quantizer step. The quantization function is a staricase function, that is $Q'(z) = \sum \delta_i$ which is zero almost everywhere so backpropagation cancels out.
To fix this n additive uniform noise is added

$$\hat{z} = Q(z) + u, \text{ where } u = \mathcal{U}(-0.5, 0.5)$$

This allows the quantization function to be differentuable and the noise is independend white noise at higher resolutions

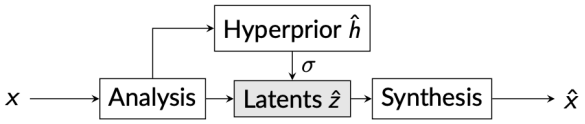
Hyperprior

The base compression can be improved by sending some additional information bits called **hyperprior**

- **Hyper-Encoder:** takes latent tensors y and extracts statistical natures z
- **Hyper-Decoder:** decodes $\hat{z}$ to predict the standard deviation of the conditional propri probability

JPEG-AI

JPEG AI uses a **scale hyperprior autoencoder**



JPEG AI Architecture

It has a innovative feature called **dual-use bitstream**. This can decode the file in two useful ways:

- Human Vision Decoder: reconstructs standard pixels for visual consumption
- Computer Vision Task Decoder: extracts features directly form the latent space without full pixel reconstruction

6) Motion Estimation

Videos are different from images as they implement temporal information. This information is mostly found in the movement. The study of **optical flow** consists in defining the movement of a pixel between two subsequent images into a **vector field**.

Optical flow consists in finding a 2D vector field $V(x, y)$:

$$V : (x, y) \in \mathscr{I} \subset \mathbb{R}^2 \to (u, v)$$

where:

- x, y are the points on the image
- $u(x, y), v(x, y)$ is the velocity of the point x, y

6.1) Variatonal Method

Consider a point of an object moving from pixel $p - D$ to p in time T. The trajectory of the pixel becomes:

$$\begin{matrix} x(t_0) = p - D \\ x(t_0 + T) = p \end{matrix} \longrightarrow D(p.\, t_0, T) = x(t_0 + T) - x(t_0) = \begin{bmatrix} c(x, y) \\ d(x, y) \end{bmatrix}$$

where c, d depend on p, t_0, T but the time parameters are ignored and $p = (x, y)$.

We can find it's derivative and find the velocity field:

$$V(x, y) = \lim_{T \to 0} \frac{D(x, y)}{T} = \begin{bmatrix} u(x, y) \\ v(x, y) \end{bmatrix}$$

Finally the OF equation is:

$$u f_x + v f_y + f_t = 0$$

with u, v components of the velocity field, f_x, f_y the space derivatives and f_t the time derivative.

This formula states that, the intensity change I see in a point depends only on the movement of the pixels.

Hypothesis 13 (Constant Illumination)**.**

We consider a continuous representation of the video signal. The Constant Illumination Hypothesis (CIH) states that the luminance does not change along the motion trajectory:

$$f(x, y, t + T) = f(x - c, y - d, t) \longrightarrow \frac{df}{dt} = 0$$

But in practice due to sampling, aliasing and noise this is not true.

We can use this hypothesis to derive the OF equation.
Apply taylor to CIH:

$$f(p, t + T) = f(p, t) - c(p) f_x(p, t) - d(p) f_y(p, t) + o(\|D(p)\|) = f(p, t) - D \nabla f + o(\|D(p)\|)$$

And now find the partial time derivative:

$$f_t \overset{T \to 0}{=} \frac{f(p, t + T) - f(p, t)}{T} = \frac{-D \cdot \nabla f}{T} + \frac{o(\|D(p)\|)}{T} = -V \nabla f + \frac{o(\|D(p)\|)}{T}$$

From here the OF equation becomes:

$$f_t = -V\nabla f \rightarrow V\nabla f + f_t = 0 \rightarrow uf_x + vf_y + f_t = 0$$

However the OF equation has 2 unknowns. We need an additional constraint to solve the problem. Also CIH isn't true in practice.

Solution: Minimize the energy of OF equation under suitable constraints.

Horn and Schunk introduced a constrain on the total variation of V over a region $\mathscr{R}$:

$$\begin{cases} \int\int_{\mathscr{R}} (uf_x + vf_y + f_t)^2 dxdy = \min \\ \int\int_{\mathscr{R}} \|\nabla u\|^2 + \|\nabla v\|^2 dxdy \leq \tau \end{cases} \rightarrow \begin{cases} u = \overline{u} - f_x \frac{\overline{u}f_x + \overline{v}f_y + f_t}{\lambda \|\nabla f\|^2} \\ v = \overline{v} - f_y \frac{\overline{u}f_x + \overline{v}f_y + f_t}{\lambda \|\nabla f\|^2} \end{cases}$$

where $\hat{\cdot}$ is the temporal average

The result is obtained via Lagrange multiplier with minimization on u and v;

$$J = \int\int_{\mathscr{R}} (uf_x + vf_y + f_t)^2 + \lambda(\|\nabla u\|^2 + \|\nabla v\|^2) dxdy$$

this is done

TODO

6.2) Block Matching Method

This method allows to use discrete signals and having as a support a block of pixels rather than a single one.

This technique is very popular as it gives good results at a low computational cost.

Consider a $P \times Q$ block of pixels inside the image:

$$B_{p,q} = \{p, p+1, \dots, p+P-1\} \times \{q, q+1, \dots, q+Q-1\}$$

And the luminance vector at time k:

$$f_k(B_{p,q}) = [f(p,q,k), \dots]^T$$

The block matching method consists in computing the dissimilarity between blocks and selecting those with minimum dissimilarity. That is

$$(\hat{i}, \hat{j}) = \arg\min_{i,j} d[f_k(B_{p,q}), f_h(B_{p-i,j-q})]$$

In general we have a **forward motion**, that is $h = k-1$ with h the current frame and k the reference frame. Therefore the OF field at frame h will show the direction in which the blocks will be at frame k.

$$\forall (n,m) \in B_{p,q}, (u_{h\rightarrow k}, v_{h\rightarrow k}) = \arg\min_{(i,j)\in\mathcal{W}} d[f_k(B_{p,q}), f_h(B_{p-i,q-j})]$$

Quality Measure

One valid quality measure is the **energy of the prediction error**, that is the MSE of the predicted block and the real block. That is:

$$e(n,m) = f_k(n,m) - \tilde{f}_k(n,m) \to \mathscr{E} = \frac{1}{NM}\sum_{n,m} e^2(n,m) \to PSNR = 10\log_{10}\frac{255^2}{\mathscr{E}}$$

where $\tilde{f}_k(n,m) = f_h(n + u_{h\to k}, m + v_{h\to k})$

Performance Factors

The three computing performance factors are:

- Block size P, Q
- Number of candidate vectors ($i, j \in \mathcal{W}$)
- Cost function d

A **large block size** reduces complexity (less blocks), less coding cost, increased MSE. The ideal block size is 16×16.

The **cost function** is based on $\mathcal{L}_1$ norm (SAD) or $\mathcal{L}_2$ norm (SSD). With SSD the one with smallest MSE but more computing cost

But a regularization term is also added, so the minimization is on J:

$$J(\mathrm{v}) = d(\mathrm{v}) + \lambda_{ME} r(\mathrm{v})$$

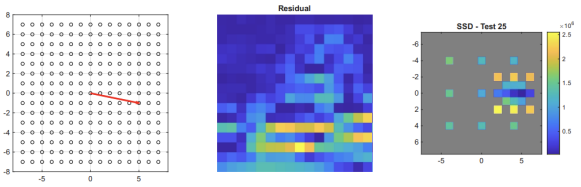
where λ_{ME} affects the importance of the regularization term.
Possible regularization terms are:

- Cost function: choose not best MSE, but best coding option.
- Distance: prioritize vectors with length same as mean of adjacent blocks

Cost Function	Pro	Con
SSD	Optimizes PSNR	Larger rate (outliers)
SAD	Better rate (implicit regularization)	Worse PSNR
SAD+regularization	Best option	

The **number of candidates** can also be reduced by using some research strategies:

- Naive: test every vector i, j
- Less naive: test every vector in a $2A + 1 \times 2B + 1$ window center in i, j
- Three Steps Search (3SS): Assumption that error function is unimodal (single global minimum and no local minimum) Test 4-8 points and choose minimum error, now divide window and search 4-8 with window centered in previous minimum repeat



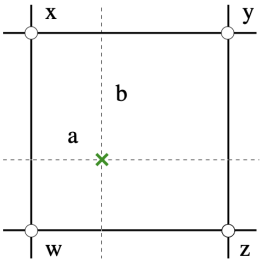
Example

- Diamond Search: search in 9 point diamond pattern, then extend pattern in direction of minima
- Hexagon Search: same as diamond but with hexagon (more modern)

- TZSearch: new technique that adaptively changes. Start with big block, if error is too big split it, repeat.

It is also possible to test sub-pixel positions by interpolation:

$$f(n+a, m+b) = (1-a)(1-b)x + a(1-b)y + (1-a)bz + abw$$



Example

Fixed Search	Unbound Search
Fixed number of steps	Iterative, faster
Guarantees global minimum	Can get stuck in local minima

6.3) Parametric Methods

A parametric model shows the motion field as a closed form function of the pixel position. The dof are the parameters of the function. It can be global or region based (local).
Block Matching is a special case of parametric methods with 2 parameters per bock (u,v translations), that is the **translational**:

$$v(p) = \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

The affine model is

$$v(p) = b + Bp = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} + \begin{bmatrix} b_3 & b_4 \\ b_5 & b_6 \end{bmatrix} p$$

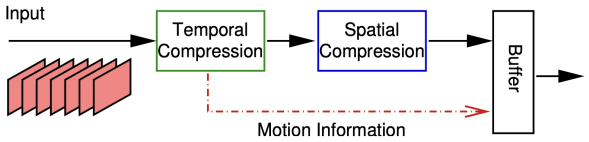
This has only 6 dof but can represent many complex fields: rotation, zoom, translation

How are the parameters of the model estimated?
First dense field, then find global motion by least squares

6.4) Deep Learning Methods TODO

7) Video Compression Principles

Video compression uses the spatial redundancy (seen in jpeg) but also time redundancy via motion fields



General Scheme

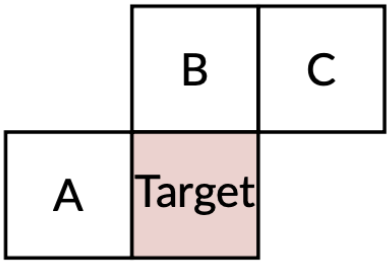
The temporal compression works by dividing the image in blocks $B_k^{(p)}$ and finding the most similar block $B_h^{(p+v)}$ in the ref image. The resulting displacements form the motion field.

The resulting vector fields are similar to the geometry of the scene, so the signal is NOT sparse. Therefore Exp-Golomb should **not** be used (not centered in 0).

We can still use predictors:
To reduce the bitrate we exploit the correlation between adjacent vectors. We define a Motion Vector Predictor (MVP) and encode only the Difference (MVD)

$$MVD = MV - MVP$$

One useful predictor is the median amongst 3 adjacent blocks:



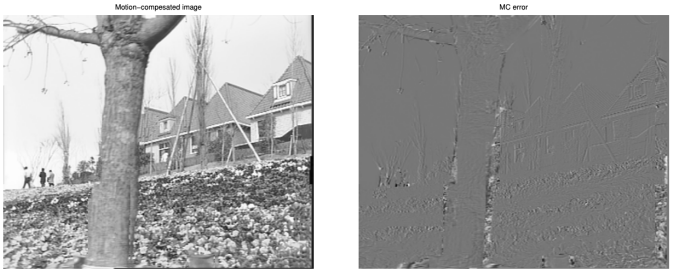
Median Blocks

This reduces substantially the number of bits needed.

Motion Compensation

We can predict the entire new image by copying the blocks in the new position:

$$\hat{I}_k(p) = I_h(p + v^*(p))$$



Example

But not all blocks should be replaced so they are signaled as intra (use og block) or inter frame blocks (use predictor).

For each $B_k^{(p)}$ do:

- Perform Motion Estimation (ME) with ref image h:

$$J(v) = d(B_k^{(p)}, B_h^{(p+v)}) + \lambda_{ME} r(v) \rightarrow v^* = \arg\min_v J(v)$$

- **Mode select**: decide if inter or intra (see later how)
  - If intra use jpeg
  - If inter encode v^* and $E(p) = B_k^{(p)} - B_h^{(p+v)}$

A variable block size is more effective although it is more complex to implement.

8) Proofs

Kraft Inequality:
Proof of Necessity ($\implies$):
Codewords are already given. Set $L_{\max} = \max_i(l_i)$

Build binary tree with depth $= L_{\max} \rightarrow 2^L_{\max}$ number of leaves.

Each codeword with prefix property occupies an entire subtree (since no other codeword must start with same bits). So a codeword with length l_i occupies $2^{L_{\max}-l_i}$ leaves $\rightarrow$ No codeword is a descendant of another, thus these sets of leaves are disjoint.

From here notice that The total number of blocked leaves cannot exceed the available leaves, formally:

$$\sum_{i=1}^{N} 2^{L_{\max}-l_i} \leq 2^{L_{\max}} \rightarrow \sum_{i=1}^{N} 2^{-l_i} \leq 1$$

Proof of Sufficiency ($\impliedby$):
We have given $\{l_i\}$ such that $\sum 2^{-l_i} \leq 1$
Again build a binary tree:
The codes can be built as follows:

1. Sort all lengths $l_1 \leq \ldots \leq l_N$
2. At each step $k = \{1, \ldots, N\}$ pick an available position at depth l_k (decreasing order)
3. Mark descendants of l_k as forbidden. 2^{-l_i} nodes are blocked.

At each step the unavailable nodes are $\sum_{i=1}^{k-1} 2^{-l_i}$. Therefore we have $1 - \sum_{i=1}^{k-1} 2^{-l_i}$ available nodes and by Kraft's Inequality we have

$$\sum_{i=1}^{N} 2^{-l_i} = \sum_{i=1}^{k-1} 2^{-l_i} + 2^{-l_k} \leq 1 \rightarrow 1 - \sum_{i=1}^{k-1} 2^{-l_i} \geq 2^{-l_k}$$

Since the remaining free capacity is at least 2^{-l_k}, there must exist at least one free node at depth l_k where the k-th codeword can be placed.

□